Design and Analysis of Algorithms

Assignment 2 — Data Structures

Student: Taubakabyl Nurlybek
Course: Design and Analysis of Algorithms
Assignment: 2 — Data Structures
Rakhym Sabyr SE-2517

---

1. Introduction

The purpose of this assignment is to implement and analyze several basic data structures without using Java's standard collection implementations.

The implemented data structures are:

- DynamicArray
- MyLinkedList
- MinHeap

All structures store primitive int values. The goal is not only to implement the required operations, but also to analyze their theoretical complexity and practical performance.

Four different workloads were used to compare the structures:

1. Random access.
2. Search.
3. Insertion and removal at the head and in the middle.
4. Priority processing using a minimum heap.

The experiments were performed for input sizes of:

- 100
- 1,000
- 10,000
- 100,000

The same random data generation procedure was used for all experiments with Random(42) to make the benchmark reproducible.

The benchmark also counts physical operations such as steps, element moves, and comparisons. This makes it possible to compare theoretical complexity with actual work performed by each data structure.

---

2. Data Structures

2.1 DynamicArray

DynamicArray is a dynamically growing array implemented using a primitive int[].

The structure maintains an internal array and the current number of stored elements.

When the array becomes full, its capacity is increased by a factor of two. The existing elements are copied into the new larger array.

The following operations are implemented:

- add(x)
- add(index, x)
- remove(index)
- get(index)
- contains(x)

Adding an element

Adding an element to the end normally requires placing the value at the next free position.

When there is no free space, the array is resized. A new array with twice the previous capacity is created and all existing elements are copied.

Therefore, although a resize itself requires linear time, normal additions are constant time.

Inserting at an index

When inserting an element at a specific index, elements from that index to the end must be shifted one position to the right.

Therefore, insertion at an arbitrary position requires linear time in the worst case.

Removing an element

When an element is removed from the middle or beginning, all following elements must be shifted one position to the left.

Therefore, removal from an arbitrary position has linear worst-case complexity.

Accessing an element

Because the structure uses an array, an element can be accessed directly using its index.

The address of an element can be calculated directly, so get(index) takes constant time.

Searching

contains(x) performs a linear search through the array until the required value is found or the end is reached.

Therefore, searching requires linear time in the worst case.

2.2 MyLinkedList

MyLinkedList is a linked-list implementation created from scratch.

Each node stores:

- an integer value;
- a reference to the next node.

The structure supports the same basic operations as DynamicArray:

- add(x)
- add(index, x)
- remove(index)
- get(index)
- contains(x)

Unlike an array, linked-list elements are not stored in one contiguous memory block.

To access an element by index, the list must start from the first node and follow links until the required position is reached.

This makes random access significantly slower than array access.

Insertion and removal

Insertion or removal at the beginning of the list is very efficient because only the first node reference has to be changed.

However, insertion or removal at an arbitrary index requires traversing the list to the required position.

Therefore, the traversal itself can take linear time.

Search

contains(x) checks nodes sequentially until the requested value is found or the end of the list is reached.

Thus, its worst-case complexity is linear.

2.3 MinHeap

MinHeap is an array-based binary minimum heap.

The heap satisfies the following property:

The value stored at every parent node is less than or equal to the values stored in its children.

The following operations are implemented:

- insert(x)
- peekMin()
- extractMin()

Insert

A new value is first placed at the end of the heap.

The value is then compared with its parent. If it is smaller than the parent, the two elements are exchanged.

This process is repeated until the heap property is restored.

This operation is called **bubble-up**.

The height of a binary heap is logarithmic in the number of elements, therefore insertion has O(log n) worst-case complexity.

Peek minimum

The minimum element is always stored at the root of the heap.

Therefore, peekMin() directly returns the first element without modifying the heap.

Its complexity is O(1).

Extract minimum

To remove the minimum element, the root is replaced with the last element of the heap.

The new root is then compared with its children. If necessary, it is exchanged with the smaller child.

This process continues until the heap property is restored.

This operation is called **bubble-down**.

Its worst-case complexity is O(log n).

---

3. Complexity Analysis

The following table summarizes the complexity of the implemented operations.

3.1 DynamicArray

Operation	Best	Average	Worst
add(x)	Ω(1)	Θ(1) amortized	O(n)
add(index, x)	Ω(1)	Θ(n)	O(n)
remove(index)	Ω(1)	Θ(n)	O(n)
get(index)	Ω(1)	Θ(1)	O(1)
contains(x)	Ω(1)	Θ(n)	O(n)

The important detail is add(x). Most additions take constant time, but an occasional resize requires copying all elements. Therefore, the amortized complexity is Θ(1).

3.2 MyLinkedList

Operation	Best	Average	Worst
add(x)	Ω(1)	Θ(1)	O(1)
add(index, x)	Ω(1)	Θ(n)	O(n)
remove(index)	Ω(1)	Θ(n)	O(n)
get(index)	Ω(1)	Θ(n)	O(n)
contains(x)	Ω(1)	Θ(n)	O(n)

The best case for indexed operations occurs when the requested position is at the beginning of the list. In general, however, a linked list must traverse nodes to reach an arbitrary index.

3.3 MinHeap

Operation	Best	Average	Worst
insert(x)	Ω(1)	Θ(log n)	O(log n)
peekMin()	Ω(1)	Θ(1)	O(1)
extractMin()	Ω(1)	Θ(log n)	O(log n)

The best case for insert occurs when the new element is already greater than or equal to its parent and therefore does not need to move upward.

Similarly, extractMin can finish quickly if the replacement element already satisfies the heap property.

---

4. Correctness and Loop Invariants

Two operations containing loops were selected for correctness analysis.

4.1 Loop Invariant for contains(x)

Consider the loop that sequentially checks elements until the required value is found.

Invariant

Before every iteration, all elements at positions before the current position have already been checked, and none of them is equal to the searched value x.

Initialization

Before the first iteration, the current position is the first element of the structure.

There are no previous elements to check. Therefore, the invariant is true.

Maintenance

Assume that the invariant is true before an iteration.

The current element is checked.

If it is equal to x, the method can immediately return true.

Otherwise, the current element is not equal to x. The loop moves to the next position.

Therefore, all elements before the new current position have been checked and none of them equals x.

The invariant is therefore maintained.

Termination

The loop terminates in one of two cases.

First, the searched value is found. In this case, the method returns true.

Second, the loop reaches the end of the structure. According to the invariant, every element has been checked and none is equal to x. Therefore, returning false is correct.

Conclusion

The loop invariant proves that contains(x) returns true exactly when the searched value exists in the structure. Therefore, the operation is correct.

4.2 Loop Invariant for MinHeap Bubble-Down

The extractMin() operation uses a bubble-down loop to restore the heap property.

After removing the root, the last element is moved to the root and may violate the heap property.

Invariant

Before every iteration of the bubble-down loop, the heap property is satisfied everywhere except possibly at the current node and its children. All subtrees below the children of the current node are already valid heaps.

Initialization

Immediately after replacing the root with the last element, the only possible violation is at the root.

All other subtrees were valid heaps before the extraction.

Therefore, the invariant is true before the first iteration.

Maintenance

During an iteration, the current node is compared with its children.

If the current node is smaller than or equal to both children, the heap property is restored and the loop terminates.

Otherwise, the current node is exchanged with its smaller child.

The smaller child becomes the new current position.

The subtree that was previously below the selected child remains a valid heap, while the only possible violation moves downward.

Therefore, the invariant remains true for the next iteration.

Termination

The loop terminates when either:

1. the current node satisfies the heap property; or
2. the current node has no children.

In both cases, there is no remaining heap-property violation.

Therefore, the entire structure is again a valid minimum heap.

Conclusion

The bubble-down loop restores the minimum-heap property after removing the root. Consequently, extractMin() returns the correct minimum element and leaves the remaining elements in a valid heap.

---

5. Benchmark Methodology

The benchmark evaluates the data structures under four workloads.

All experiments use the same input generation strategy based on:

new Random(42)

Using a fixed seed makes the experiments reproducible.

The tested input sizes are:

n = 100

n = 1,000

n = 10,000

n = 100,000

The benchmark records:

- execution time;
- number of physical steps;
- number of element moves;
- number of comparisons.

This provides both practical timing information and a more direct measurement of the amount of work performed by each data structure.

---

6. Workload W1 — Random Access

The first workload evaluates indexed access.

Each structure is filled with n values. Then 10,000 get(index) operations are performed using randomly selected indices.

Only DynamicArray and MyLinkedList are compared.

The theoretical expectation is:

- DynamicArray.get(index) — O(1)
- MyLinkedList.get(index) — O(n)

The benchmark confirms this difference.

For DynamicArray, the number of recorded steps remains approximately constant at 10,000 regardless of the input size.

For MyLinkedList, the number of steps grows approximately proportionally to n:

n	DynamicArray time (ms)	MyLinkedList time (ms)
100	0.6114	1.2466
1,000	0.1575	9.5454
10,000	0.1822	99.6709
100,000	0.0381	761.1848

The linked list becomes much slower as the input size increases because each indexed access requires traversing nodes.

The array can calculate the memory position of an element directly.

---

7. Workload W2 — Search

The second workload evaluates the contains(x) operation.

A total of 1,000 search queries are performed, with approximately half of the queries targeting values that are present and half targeting values that are absent.

Both structures perform a linear search.

Therefore, the theoretical worst-case complexity is:

DynamicArray: O(n)

MyLinkedList: O(n)

However, the benchmark demonstrates that the two structures do not have identical running times.

n	DynamicArray time (ms)	MyLinkedList time (ms)
100	0.6477	0.6679
1,000	0.7624	1.7137
10,000	6.2350	16.7737
100,000	34.6603	119.9482

Although both structures have the same asymptotic complexity, DynamicArray is consistently faster.

The main reason is memory locality.

The elements of DynamicArray are stored in contiguous memory. When the processor reads one element, nearby elements are likely to already be available in the CPU cache.

In contrast, nodes of MyLinkedList can be located in different memory locations. Following the next reference requires pointer chasing and results in poorer cache locality.

This demonstrates that Big-O complexity does not completely determine practical performance.

---

8. Workload W3 — Insert and Remove

The third workload evaluates repeated insertions and removals.

Two variants are tested:

- operations at the head;
- operations in the middle.

Each variant performs 1,000 insertions and 1,000 removals.

8.1 W3 Head

For operations at index 0, the results are:

n	DynamicArray time (ms)	MyLinkedList time (ms)
100	2.0370	0.1657
1,000	0.5131	0.1017
10,000	2.4781	0.0142
100,000	17.5956	0.0104

The linked list performs extremely well for operations at the head because insertion and removal can be performed by changing the first node reference.

For DynamicArray, inserting or removing at index 0 requires shifting many existing elements.

The recorded move counts demonstrate this difference.

For example, at n = 100,000, the DynamicArray performs approximately 201 million moves during the workload, while the linked list performs no element moves.

Therefore, a linked list is much more suitable for frequent boundary modifications at the head.

---

9. W3 Middle

For middle operations, the situation changes.

n	DynamicArray time (ms)	MyLinkedList time (ms)
100	0.3716	1.0233
1,000	0.5111	2.0223
10,000	1.5173	20.2885
100,000	8.8969	153.3619

Both data structures have linear complexity for arbitrary middle insertion and removal.

However, DynamicArray is substantially faster in practice.

The reason is that the array stores elements contiguously. Although many elements have to be shifted, these operations are performed on contiguous memory.

For MyLinkedList, the program must first traverse the list to reach the middle position. The traversal follows individual pointers, which creates pointer chasing and poor cache locality.

This is an important example of why two algorithms with the same Big-O complexity can have very different real execution times.

---

10. Workload W4 — Priority Processing

The fourth workload evaluates MinHeap.

The heap is filled with n values. Then extractMin() is called n times.

The returned values are checked to make sure they are in non-decreasing order.

The measured execution times are:

n	MinHeap time (ms)
100	0.3120
1,000	0.6025
10,000	1.9899
100,000	11.9093

The complete workload consists of n insertions and n extractions.

Since each heap insertion and extraction has logarithmic worst-case complexity, the complete workload has:

O(n log n)

complexity.

The benchmark shows that the number of physical operations grows faster than linearly but significantly slower than quadratic growth, which is consistent with the expected O(n log n) behavior.

The extraction results were also checked to ensure that the returned values are in non-decreasing order. This verifies the correctness of the priority-processing workload.

---

11. Physical Operation Counts

In addition to execution time, the benchmark records physical operations.

These measurements are useful because execution time can be affected by many external factors, including the Java Virtual Machine, CPU frequency, operating-system scheduling, and cache state.

Operation counts provide a more direct view of the algorithmic work.

For example, during W3 head operations with n = 100,000, DynamicArray performs approximately:

201,000,000 steps

200,999,000 moves

while MyLinkedList performs approximately:

0 steps

3,000 moves

for the corresponding counters.

This difference is consistent with the underlying algorithms.

The DynamicArray must repeatedly shift elements, whereas the linked list changes references instead of moving large blocks of values.

For W4 with n = 100,000, the MinHeap records approximately:

9,505,680 steps

3,287,430 moves

3,059,125 comparisons

These values reflect the repeated bubble-up and bubble-down operations required by the heap.

---

12. Cache Locality and Pointer Chasing

One of the most important observations from the benchmark is that theoretical complexity does not completely determine real execution time.

DynamicArray stores elements consecutively in memory.

This provides good **cache locality**.

When the CPU accesses one element, nearby elements are likely to be loaded into the cache as well. Sequential operations can therefore be performed efficiently.

MyLinkedList has a different memory layout.

Each node contains a value and a reference to another node. The nodes are not necessarily located next to each other in memory.

When the program follows the next reference, the processor may have to access a different memory location.

This is called **pointer chasing**.

Pointer chasing can cause more cache misses and memory latency.

Therefore, two operations can both have O(n) complexity while having very different execution times.

This explains why DynamicArray.contains() is faster than MyLinkedList.contains() even though both have linear complexity.

---

13. Big-O Versus Real Running Time

The experiments demonstrate that Big-O notation describes how an algorithm scales with input size, but it does not describe every detail of its real execution.

For example, both DynamicArray.contains() and MyLinkedList.contains() have:

O(n)

worst-case complexity.

Nevertheless, at n = 100,000, the measured times were:

DynamicArray: 34.6603 ms

MyLinkedList: 119.9482 ms

Therefore, the linked list took substantially longer.

Similarly, both middle insertion operations have linear complexity, but the DynamicArray was considerably faster in the benchmark.

The reason is that actual performance depends on factors such as:

- memory layout;

- CPU cache locality;
- pointer chasing;
- number of memory accesses;
- number of element movements;
- branch behavior;
- JVM and hardware effects.

Consequently, asymptotic complexity and physical operation counts should be considered together when analyzing a data structure.

---

14. Trade-offs Between Data Structures

Each data structure has advantages and disadvantages.

DynamicArray

Advantages

- Constant-time indexed access.
- Good cache locality.
- Efficient sequential traversal.
- Efficient search compared with a linked list in practice.
- Low memory overhead per element.

Disadvantages

- Insertion at the beginning or middle requires shifting elements.
- Removal at the beginning or middle requires shifting elements.
- Resizing requires copying elements.

MyLinkedList

Advantages

- Very efficient insertion and removal at the head.
- No need to shift large numbers of elements.
- The structure can grow without reallocating a contiguous array.

Disadvantages

- Indexed access is linear.
- Search is linear.
- Poor cache locality.
- Pointer chasing increases practical running time.
- Each node requires additional memory for references.

MinHeap

Advantages

- Constant-time access to the minimum element.
- Efficient insertion.
- Efficient extraction of the minimum.
- Suitable for priority-based scheduling.

Disadvantages

- Does not provide constant-time arbitrary search.
- Elements are not stored in sorted order.
- Maintaining the heap property requires additional comparisons and movements.

---

15. Conclusion

This assignment implemented three fundamental data structures from scratch: DynamicArray, MyLinkedList, and MinHeap.

The theoretical analysis and experimental results demonstrate the expected complexity characteristics.

DynamicArray is the best choice for workloads requiring frequent random access because get(index) takes constant time and the elements are stored contiguously.

MyLinkedList is more suitable for frequent insertion and removal at the beginning because these operations can be performed without shifting a large number of elements.

For middle operations, however, DynamicArray performed better in the benchmark because contiguous memory and cache locality compensate for the cost of shifting elements.

MinHeap provides an efficient solution for priority processing. peekMin() takes constant time, while insertion and extraction require logarithmic time.

The benchmark also demonstrates an important practical lesson: identical Big-O complexity does not necessarily mean identical real-world performance. Memory layout, CPU cache locality, pointer chasing, and the number of physical operations can have a significant effect on execution time.

Overall, the experiments show why both theoretical complexity and practical hardware-related effects should be considered when selecting a data structure for a particular workload.

---

16. Final Summary Table

Structure	Operation	Best	Average	Worst
DynamicArray	get(index)	O(1)	O(1)	O(1)
DynamicArray	contains(x)	O(1)	O(n)	O(n)
DynamicArray	add(x)	O(1)	O(1) amortized	O(n)
DynamicArray	add(index,x)	O(1)	O(n)	O(n)
DynamicArray	remove(index)	O(1)	O(n)	O(n)
MyLinkedList	get(index)	O(1)	O(n)	O(n)
MyLinkedList	contains(x)	O(1)	O(n)	O(n)
MyLinkedList	add(x)	O(1)	O(1)	O(1)
MyLinkedList	add(index,x)	O(1)	O(n)	O(n)
MyLinkedList	remove(index)	O(1)	O(n)	O(n)
MinHeap	insert(x)	O(1)	O(log n)	O(log n)
MinHeap	peekMin()	O(1)	O(1)	O(1)
MinHeap	extractMin()	O(1)	O(log n)	O(log n)